

INDICE

INTRODUCCION	1
CONCEPTOS Y HERRAMIENTAS USADAS.....	2
REQUISITOS MINIMOS Y ENTORNO DE DESARROLLO	4
Requisitos de Hardware (Mínimos)	4
Requisitos de Software	4
FUENTES DE DESCARGA	5
Descarga de Java JDK y Netbeans.....	5
Descarga de Graphviz y Dot	6
GUIA DE EJECUCIÓN.....	7
AFD (AUTOMATA FINITO DETERMINISTA).....	9
Alfabeto (clase de símbolos).....	10
Estados	10
Tabla de transiciones δ	11
5-tupla	13
Grafo AFD Manual	14
Grafo AFD con Graphviz y Dot.....	15
Clase GenerarDot.....	15
Clase TablaTransiciones.....	17
GRAMÁTICA REGULAR.....	20

Gramática Regular	20
Notación:	20
Producciones	21
Gramática Regular 4-tupla	23
ARQUITECTURA DEL PROYECTO	23
Paquetes:	24
BLOQUES DE CODIGO	25
MANEJO DEL ERROR	32
Categorías de error que el analizador detecta	32
DIAGRAMA UML	33
CONCLUSION	36

MANUAL TECNICO

INTRODUCCION

El presente manual técnico documenta el desarrollo del analizador léxico para el lenguaje PromptZal, elaborado como parte del Proyecto 1 del curso de Lenguajes Formales y de Programación. PromptZal es un lenguaje diseñado para estructurar prompts destinados a modelos de inteligencia artificial, organizados en agentes que agrupan un contexto, variables y comandos.

Este proyecto retoma el analizador léxico construido durante la Práctica 1 y lo eleva a un nivel más riguroso y profesional. Mientras que la práctica se enfocó en lograr un tokenizador funcional, el proyecto exige que ese reconocimiento esté respaldado por un fundamento formal: un autómata finito determinista (AFD) diseñado explícitamente para el lenguaje, junto con su gramática regular equivalente. Asimismo, se incorporan mejoras como la recuperación de errores léxicos, una interfaz gráfica para operar el analizador, y la generación de reportes que documentan los resultados del análisis.

Es importante señalar que el alcance de este trabajo continúa siendo únicamente léxico. El analizador se encarga de identificar y clasificar los distintos elementos del lenguaje como directivas, palabras reservadas, comandos, conectores, literales, entre otros, pero no interpreta su significado ni valida la estructura sintáctica del programa.

El propósito de este documento es presentar de manera clara el diseño y la fundamentación teórica detrás del analizador implementado. Para ello, se describe en primer lugar el autómata finito determinista construido para el reconocimiento de tokens, incluyendo sus estados, transiciones y su representación gráfica.

CONCEPTOS Y HERRAMIENTAS USADAS

Java: Java es un lenguaje de programación de alto nivel, orientado a objetos y diseñado para tener las menores dependencias de implementación posibles. Su filosofía *"Write Once, Run Anywhere"* (Escribe una vez, ejecútalo en cualquier lugar) permite que este proyecto sea portable.

Para este desarrollo se utilizó específicamente el JDK (Java Development Kit) versión 21, que es el conjunto de herramientas necesarias para compilar y convertir el código fuente (.java) en bytecode ejecutable por la JVM (Java Virtual Machine).

Java Swing: Java Swing es una biblioteca de Java para crear interfaces gráficas de usuario (GUI) en aplicaciones de escritorio.

Forma parte de Java Foundation Classes (JFC) y está incluida en el JDK desde Java 1.2, por lo que no requiere instalación adicional.

Apache NetBeans: Apache NetBeans es un Entorno de Desarrollo Integrado (IDE) de código abierto. Se eligió esta herramienta porque proporciona un buen soporte para la gestión de proyectos basados en Maven, facilitando la depuración, la organización de paquetes y la compilación automatizada del código. NetBeans actúa como el ecosistema donde se integran el editor de texto, el compilador y las herramientas de salida de consola.

Maven: Apache Maven es una potente herramienta de gestión y automatización de compilación, utilizada principalmente en proyectos Java. Facilita la estructura del proyecto, gestiona dependencias (bibliotecas externas) automáticamente y estandariza el ciclo de vida del

desarrollo (compilación, pruebas, empaquetado) mediante un archivo de configuración central llamado pom.xml.

Pom.xml: POM significa Project Object Model (Modelo de Objetos de Proyecto) y es el archivo de configuración central (pom.xml) en proyectos Java gestionados con Apache Maven. Define los metadatos del proyecto, gestiona las dependencias (librerías externas), complementos (plugins) y la configuración de construcción (build) para compilar, probar y empaquetar la aplicación de forma automática.

AFD: modelo matemático que procesa cadenas de símbolos paso a paso y decide si acepta o rechaza cada secuencia según las reglas de transición.

Gramática de un lenguaje regular: gramática regular es un tipo de regla matemática que sirve para generar un lenguaje regular, que es el grupo más sencillo de palabras dentro de la teoría de la computación.

Jerarquía de Chomsky: La jerarquía de Chomsky es una clasificación de cuatro niveles de las gramáticas formales que generan lenguajes formales, descrita por el lingüista Noam Chomsky en 1956.

Graphviz y Dot: Graphviz es un programa de código abierto que sirve para crear diagramas y redes a partir de instrucciones de texto, mientras que DOT es el lenguaje de programación en formato de texto plano que se utiliza para describir esos gráficos

Git: Git es un programa que se instala en la computadora para guardar el historial de los cambios dentro del código.

GitHub: GitHub es una página web en la nube que guarda copias de proyectos para compartirlos con otras personas.

REQUISITOS MINIMOS Y ENTORNO DE DESARROLLO

Para asegurar la correcta ejecución y posible modificación del sistema, el entorno debe cumplir con las siguientes especificaciones técnicas.

Requisitos de Hardware (Mínimos)

- **Procesador:** Arquitectura x64 de 2.0 GHz o superior.
- **Memoria RAM:** 8 GB (Recomendado para el manejo de IDEs pesados).
- **Almacenamiento:** 1 GB de espacio libre para el proyecto y dependencias de Maven.
- **Periféricos:** Teclado y monitor (Interfaz basada en consola).
- **Resolución de pantalla mínima:** resolución mínima de 1280x720, para no tener problemas con la interfaz gráfica.

Requisitos de Software

- **Sistema Operativo:** Windows 10/11, distribuciones de Linux (Ubuntu/Debian) o macOS.
- **Java Development Kit (JDK):** Versión 21 (LTS).
- **IDE:** Apache NetBeans 21 o superior.
- **Graphviz :** Versión recomendada (Graphviz 9.0 o superior).

- **Apache Maven:** al tener Apache NetBeans KIT versión 21, ya lo tendrá incorporado.
- **Git:** versión de control, necesario para el historial de commits.
- **Navegador web:** para visualizar los reportes HTML generados (tokens, errores, estadísticas), no importa cuál.
- **Sistema de visualización de imágenes:** Sistema que la computadora tiene por defecto para visualizar imágenes, no importa cual es, esto es importante para poder ver el AFD que estará en este proyecto.

FUENTES DE DESCARGA

Descarga de Java JDK y Netbeans

1. **Verificar la instalación previa de Java (JDK):** NetBeans requiere un Kit de Desarrollo de Java (JDK) compatible para ejecutarse. El usuario debe comprobar si ya cuenta con una versión adecuada (por ejemplo, JDK 17 o superior) o descargarla previamente desde sitios como Oracle o Eclipse Adoptium (Temurin)
JDK 21: <https://www.oracle.com/java/technologies/downloads/>.
2. **Acceder al sitio web oficial:** El usuario debe ingresar al portal oficial de Apache NetBeans Java **Apache NetBeans:**
<https://netbeans.apache.org/front/main/download/>, desde cualquier navegador web.
3. **Ir a la sección de descargas:** En la página principal, se debe hacer clic en el botón o pestaña de Download para visualizar la versión estable más reciente disponible.

4. **Seleccionar el instalador para el sistema operativo:** Dentro de la sección de instaladores binarios, el usuario debe elegir el archivo correspondiente a su plataforma: Instalador .exe para Windows, Archivo .pkg o .dmg para macOS, Script .sh o paquete .deb / .rpm para distribuciones de Linux.
5. **Elegir un servidor espejo (Mirror):** El sitio web sugerirá un enlace de descarga sugerido por Apache; el usuario debe hacer clic en el enlace principal para iniciar la transferencia del archivo instalador.
6. **Ejecutar el instalador:** Una vez completada la descarga, el usuario debe abrir el archivo ejecutable con permisos de administrador y seguir las instrucciones del asistente en pantalla para configurar la ruta de instalación y vincular la ruta del JDK correspondiente.

Descarga de Graphviz y Dot

1. **Acceder al portal oficial de Graphviz** El usuario debe ingresar a la página web oficial (graphviz.org) y dirigirse a la pestaña de **Download** en el menú superior.
2. **Seleccionar el instalador según el sistema operativo** Dentro de la sección de descargas, el usuario debe ubicar su plataforma:

Windows: Descargar el paquete instalador ejecutable (generalmente graphviz-install-X.X.X-win64.exe).

Linux (Debian/Ubuntu): Se puede instalar directamente desde la terminal mediante el gestor de paquetes ejecutando `sudo apt install graphviz`.

macOS: Se puede instalar fácilmente utilizando Homebrew con el comando `brew install graphviz`.

3. **Ejecutar el asistente de instalación (en Windows)** El usuario debe iniciar el archivo .exe descargado y seguir las indicaciones en pantalla para aceptar la licencia y definir el directorio de instalación.
4. **Habilitar la adición al PATH durante la instalación** En el asistente de Windows, es crucial que el usuario marque la casilla que indica **"Add Graphviz to the system PATH for all users"** (o para el usuario actual). Esto permitirá ejecutar el comando dot desde cualquier terminal o programa externo.
5. **Configurar el PATH manualmente (si no se marcó en el paso anterior)** Si el comando no se reconoce de forma automática, el usuario debe:

Abrir las **Variables de entorno del sistema** en el panel de control.

Editar la variable Path en las variables de entorno.

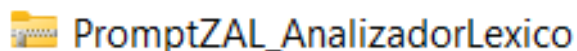
Añadir la ruta completa a la carpeta bin de Graphviz (por ejemplo, C:\Program Files\Graphviz\bin).

6. **Comprobar la instalación de la herramienta dot** Para verificar que la herramienta esté lista para su uso, el usuario debe abrir una terminal (PowerShell, CMD o bash) y ejecutar: `dot -V` El comando debe imprimir en pantalla la versión instalada de Graphviz.

GUIA DE EJECUCIÓN

Con los requerimientos mínimos y las herramientas necesarias instaladas ya es posible ejecutar el programa PROMPT_ZAL en el equipo sin ningún tipo de problema a continuación se muestra el paso a paso de como poder ejecutar el proyecto correctamente:

Con el archivo .zip se extrae en cualquier carpeta dentro del sistema preferiblemente en descargas, documentos o alguna carpeta especial.



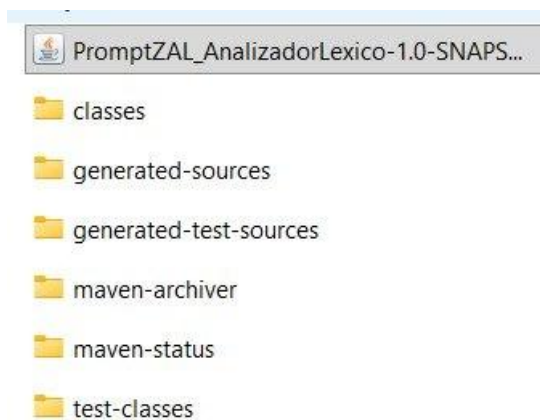
Es importante descomprimir la carpeta para poder acceder al contenido de la carpeta donde se encuentra el código fuente y el proyecto.

Archivos Pueba

src

target

En la carpeta target se encuentra el proyecto, al acceder a la carpeta es posible visualizar el archivo .jar que contiene el programa, para acceder a el simplemente se puede hacer click dos veces sobre el archivo o también se puede acceder desde la terminal.



Para acceder con terminal se debe escribir CMD, en la ruta de la carpeta una vez escrito se presiona enter y se abrirá la terminal del equipo, escribiendo la siguiente ruta: ***java -jar***

Alfabeto (clase de símbolos)

Clase	Símbolo
L	Letra o guion bajo (a-z, A-Z, _)
D	Digito (0-9)
@, " , = , + , - , > , / , * , . , { , } , (,) ,	Símbolos literales individuales
esp	Espacio, TAB
\n	Salto de línea
Otro	Cualquier carácter no reconocido
EOF	Fin del archivo

Estados

Estado	Token	Token si acepta
q0	Inicial	—
q1	Aceptación	IDENTIFICADOR
q2	Intermedio	—
q3	Aceptación	DIRECTIVA
q4	Aceptación	ENTERO
q5	Intermedio	—
q6	Aceptación	DECIMAL
q7	Intermedio	—
q8	Aceptación	CADENA
q9	Aceptación	OP_ASIGNACION(=)

q15	q15	q 1 5	q 1 5	q 1 5	q 1 5	q 1 5	q 1 5	q 1 5	q 1 5	q 1 6	q 1 5	q 1 5	q 1 5	q 1 5	q 1 5	q 1 5	q1 5
q16	q15	q 1 5	q 1 5	q 1 5	q 1 5	q 1 5	q 1 5	q 0 5	q 1 6	q 1 5	q 1 5	q 1 5	q 1 5	q 1 5	q 1 5	q 1 5	q1 5
q17 – q20	<i>(accept a de inmed iato, retorn a a q0)</i>																

5-tupla

$$M = (Q, \Sigma, \delta, q_0, F)$$

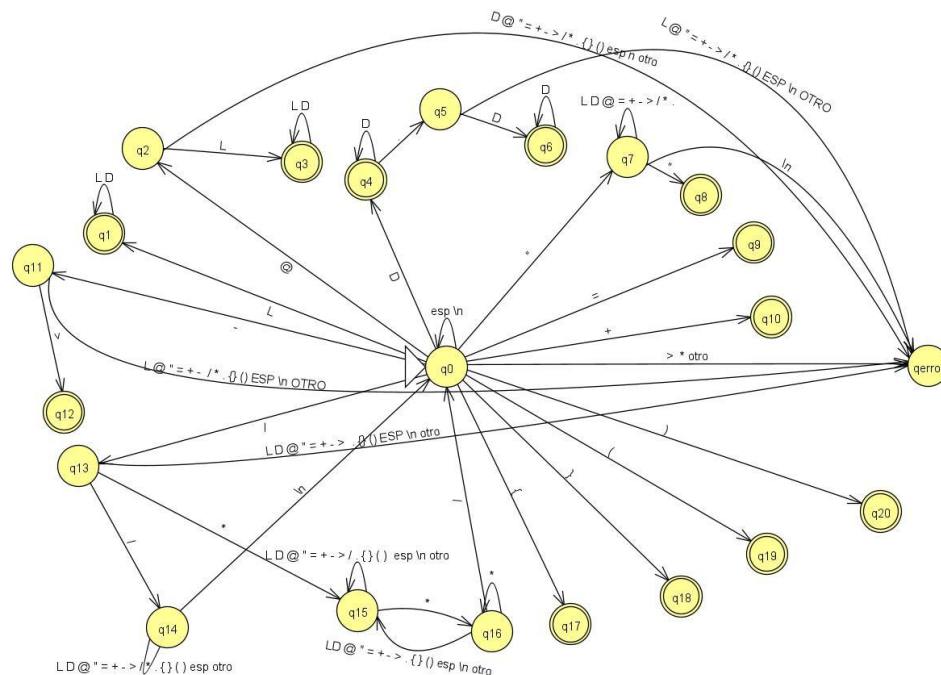
$$Q = \{q_0, q_1, q_2, \dots, q_{20}, q_{err}\}$$

$$\Sigma = \{L, D, @, ", =, +, -, >, /, *, ., \{, \}, (,), \text{esp}, \backslash n, \text{otro}\}$$

$$\delta = [\text{la tabla de transiciones completa}]$$

$$q_0 = q_0$$

$$F = \{q_1, q_3, q_4, q_6, q_8, q_9, q_{10}, q_{12}, q_{17}, q_{18}, q_{19}, q_{20}\}$$



Grafo AFD con Graphviz y Dot

La lógica del generador de código DOT se basa en traducir una estructura de datos dinámica, específicamente una tabla de transiciones, a un formato estático que el motor de Graphviz interpreta para renderizar el grafo de un autómata finito determinista., para la generación del grafo AFD se usaron dos clases GeneradorDot y TablaTransiciones

Clase GenerarDot

En GenerarDot el proceso lógico se estructura en cuatro fases principales.

La primera fase es la configuración del entorno y del motor de renderizado. El código establece reglas generales sobre como el motor debe calcular las posiciones. La instrucción para que la dirección del rango sea de izquierda a derecha es fundamental en la teoría de autómatas,

ya que ordena jerárquicamente los nodos para reflejar la lectura natural de una cadena de caracteres.

```
public String generarDot() {
    StringBuilder sb = new StringBuilder();
    sb.append("digraph AFD_PromptZal {\n");
    sb.append("    rankdir=LR;\n");
    sb.append("    nodesep=0.8;\n");
    sb.append("    ranksep=1.2;\n");
    sb.append("    dpi=200;\n");
    sb.append("    concentrate=true;\n");

    // lineas rectas sin curvas
    sb.append("    splines=polyline;\n");
}
```

La segunda fase consiste en la declaración de estilos globales. El programa define atributos estándar para los nodos y las aristas desde el principio, asignando formas circulares, fondos blancos y tipografías uniformes.

La tercera fase aborda el mapeo de los nodos, representando los estados funcionales del analizador lexico. El algoritmo evalúa la tabla para dibujar cada estado según su naturaleza. Los estados de aceptación se configuran recorriendo la estructura de datos correspondiente para cambiar su forma a un doble círculo y concatenar el nombre del token que reconocen, mientras que el estado de error se instancia independientemente para garantizar su existencia grafica.

```
// Estado de error
sb.append("    ").append(tabla.getEstadoError())
.append(" [shape=circle];\n\n");

// Transiciones
List<TablaTransiciones.Transicion> transiciones = tabla.getTransiciones();
for (TablaTransiciones.Transicion t : transiciones) {
    sb.append("    ").append(t.origen)
.append(" -> ").append(t.destino)
.append(" [label=\"").append(t.simbolo).append(" \"]");

    // Las flechas de retorno a q0
    if (t.simbolo.equals("fin token")) {
        sb.append(", style=solid, color=gray50, fontcolor=gray50");
    }

    sb.append("];\n");
}
```

La cuarta fase ejecuta el enrutamiento de las aristas, correspondientes a las transiciones. El sistema avanza sobre la lista de transiciones para conectar los nodos de origen y destino, etiquetando el vector con el símbolo que activa el cambio de estado. El aspecto más crítico de esta lógica es la manipulación de las reglas de jerarquía de Graphviz para las transiciones que indican el fin de un token y retornan al estado inicial. Dado que el motor asume por defecto que un nodo destino debe ubicarse a la derecha de su origen, las flechas de retorno deformarían el análisis visual. Para solucionarlo, el programa desactiva la restricción estructural exclusivamente para estas aristas específicas, ordenando al motor que dibuje la línea de regreso, pero sin alterar el flujo direccional del autómata.

```
// Transiciones
List<TablaTransiciones.Transicion> transiciones = tabla.getTransiciones();
for (TablaTransiciones.Transicion t : transiciones) {
    sb.append(" ").append(t.origen)
      .append(" -> ").append(t.destino)
      .append(" [label=\" ").append(t.simbolo).append(" \");

    // Las flechas de retorno a q0
    if (t.simbolo.equals("fin token")) {
        sb.append(", style=solid, color=gray50, fontcolor=gray50");
    }
}
```

Clase TablaTransiciones

La clase TablaTransiciones funciona como el modelo matemático y estructural estático de un autómata finito determinista diseñado para un analizador léxico. Su propósito central es establecer la función de transición que dicta el movimiento del escáner, definiendo el alfabeto mediante abstracciones léxicas en lugar de caracteres individuales y trazando el conjunto de estados disponibles. Internamente, encapsula las conexiones de origen y destino mediante una estructura de datos limpia, gestiona el reinicio automático del flujo hacia el estado inicial tras la

aceptación de un token para permitir un análisis continuo, y dirige sistemáticamente las entradas no válidas hacia un estado de error dedicado para evitar bloqueos en la ejecución del programa.

```

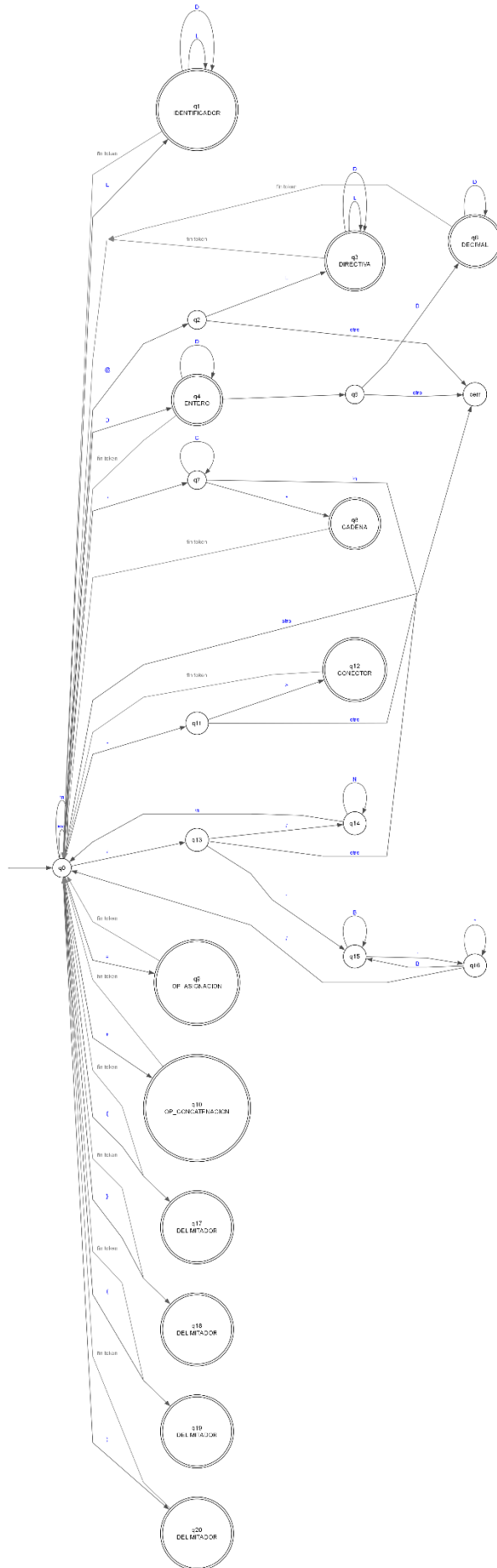
15 public class TablaTransiciones {
16     public static class Transicion {
17         public final String origen;
18         public final String simbolo;
19         public final String destino;
20
21         public Transicion(String origen, String simbolo, String destino) {
22             this.origen = origen;
23             this.simbolo = simbolo;
24             this.destino = destino;
25         }
26     }
27
28     private final List<Transicion> transiciones;
29     private final Map<String, String> aceptacion; // estado -> token que emite
30     private final String estadoInicial;
31     private final String estadoError;
32
33     public TablaTransiciones() {
34         this.transiciones = new ArrayList<>();
35         this.aceptacion = new LinkedHashMap<>();
36         this.estadoInicial = "q0";
37         this.estadoError = "qerr";
38         construir();
39     }
40

```

```

private void construir() {
    // Identificadores / palabras reservadas / comandos / conectores de palabra
    agregar("q0", "L", "q1");
    agregar("q1", "L", "q1");
    agregar("q1", "D", "q1");
    aceptar("q1", "IDENTIFICADOR");
}

```



GRAMÁTICA REGULAR

El lenguaje léxico de Prompt_Zal es un lenguaje regular (**Tipo 3 en la jerarquía de Chomsky**), es el nivel más simple de la jerarquía, y es exactamente el tipo de lenguaje que un AFD puede reconocer.

Gramática Regular

Es una gramática regular lineal derecha (todas las producciones tienen la forma $A \rightarrow aB$ o $A \rightarrow a$), que es la forma gramatical formalmente equivalente a un AFD.

Símbolo inicial: Q_0

Notación:

$L = |a, b, c, d, e, f, g, h, i, j, k, l, m, n, o, p, q, r, s, t, u, v, w, x, y, z|, |A, B, C, D, E, F, G, H, I, J, K, L, M, N, O, P, Q, R, S, T, U, V, W, X, Y, Z, _ |$

$D = |1, 2, 3, 4, 5, 6, 7, 8, 9, 0|$

$C =$ cualquier carácter excepto “y \n

$N =$ cualquier carácter excepto \n

$B =$ cualquier carácter excepto *

Identificadores, palabras reservadas, comandos y conectores de palabra

$Q_0 \rightarrow L Q_1$

$Q_0 \rightarrow \text{esp } Q_0 \mid \backslash n Q_0$

$$Q1 \rightarrow L Q1 \mid D Q1 \mid \varepsilon$$

Producciones

Directivas

$$Q0 \rightarrow @ Q2$$

$$Q2 \rightarrow L Q3$$

$$Q3 \rightarrow L Q3 \mid D Q3 \mid \varepsilon$$

Enteros y decimales

$$Q0 \rightarrow D Q4$$

$$Q4 \rightarrow D Q4 \mid . Q5 \mid \varepsilon$$

$$Q5 \rightarrow D Q6$$

$$Q6 \rightarrow D Q6 \mid \varepsilon$$

Cadenas

$$Q0 \rightarrow " Q7$$

$$Q7 \rightarrow C Q7 \mid " Q8$$

$$Q8 \rightarrow \varepsilon$$

Operadores

$$Q0 \rightarrow = Q9 \quad Q9 \rightarrow \varepsilon$$

$Q_0 \rightarrow + Q_{10} \quad Q_{10} \rightarrow \varepsilon$

Conector flecha

$Q_0 \rightarrow - Q_{11}$

$Q_{11} \rightarrow > Q_{12}$

$Q_{12} \rightarrow \varepsilon$

Delimitadores

$Q_0 \rightarrow \{ Q_{17} \quad Q_{17} \rightarrow \varepsilon$

$Q_0 \rightarrow \} Q_{18} \quad Q_{18} \rightarrow \varepsilon$

$Q_0 \rightarrow (Q_{19} \quad Q_{19} \rightarrow \varepsilon$

$Q_0 \rightarrow) Q_{20} \quad Q_{20} \rightarrow \varepsilon$

Comentarios

$Q_0 \rightarrow / Q_{13}$

$Q_{13} \rightarrow / Q_{14} \mid * Q_{15}$

$Q_{14} \rightarrow N Q_{14} \mid \backslash n Q_0 \mid$

$Q_{15} \rightarrow B Q_{15} \mid * Q_{16}$

$Q_{16} \rightarrow / Q_0 \mid * Q_{16} \mid B Q_{15}$

Gramática Regular 4-tupla

$$G = (N, T, P, S)$$

$$N = \{Q_0, Q_1, Q_2, \dots, Q_{20}\}$$

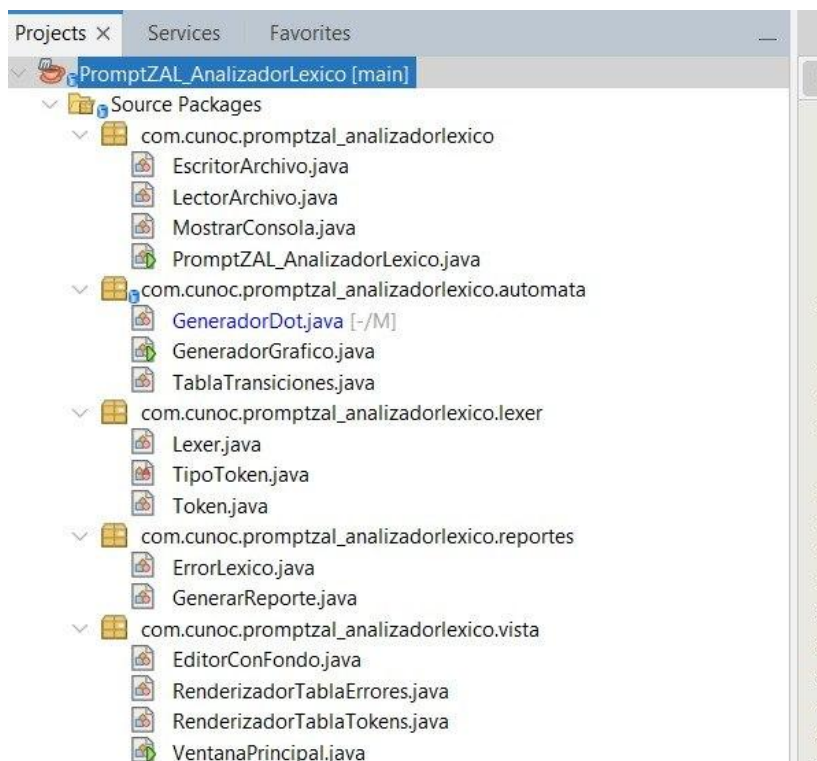
$$T = \{L, D, @, ", =, +, -, >, /, *, ., \{, \}, (,), C, N, B, \text{esp}, \backslash n\}$$

$$P = [\text{todas las producciones ya escritas}]$$

$$S = Q_0$$

ARQUITECTURA DEL PROYECTO

La estructura del proyecto se encuentra distribuida de la siguiente manera para mantener un orden así evitar sobre cargar los paquetes o las clases y mejorar la eficiencia del código fuente, el código fuente se encuentra adjuntado en la carpeta del proyecto.



Paquetes:

Distribución del código en paquetes para no mezclar funciones.

com.cunoc.promptzal_analizadorlexico	Paquete main, aquí se encuentra la clase que arranca todo el proyecto, además del lector y escritor de archivos necesarios para el manejo de archivos. pz del Analizador Léxico.
com.cunoc.promptzal_analizadorlexico.automata	En este paquete se encuentran las tres clases que le dan forma al grafo del AFD, junto con el código DOT para generarlo como una imagen.

com.cunoc.promptzal_analizadorlexico.lexer	El paquete lexer contiene la clase más importante Lexer donde se encuentra la lógica que clasifica y reconoce los tokens.
com.cunoc.promptzal_analizadorlexico.reportes	El paquete reportes contiene el manejo de errores en los htmls además de la generación del los 3 reportes(tokens, errores y estadísticas)
com.cunoc.promptzal_analizadorlexico.vista	Vista es el paquete con toda la interfaz gráfica, usando java swing y las herramientas disponibles como (JFram , JTable, JButton, JText , JfileChooser por mencionar algunas), en este paquete esta trabajado todo lo relacionado a la parte que el usuario ve y con la que interactúa.

BLOQUES DE CODIGO

En esta sección se resaltan las partes del código mas importantes para este proyecto 1, cabe resaltar que los bloques de código que estarán en esta parte, son bloques de código nuevo, debido a que el proyecto 1 esta conectado a la practica 1, muchos bloques de código ya fueron definidos en el anterior manual porque lo que se recomienda leer el manual de la practica 1.

Paquete Mostrar Consola.

Clase lector Archivo

```

14 public class LectorArchivo {
15     //es el encargado de leer el archivo y guardarlo mientras tanto en la ram
16     public static String leer(String ruta){
17         StringBuilder sb = new StringBuilder();
18
19         try(BufferedReader br = new BufferedReader(new FileReader(ruta))){
20             String linea;
21             while ((linea = br.readLine()) != null){
22                 sb.append(linea).append("\n");
23             }
24         } catch (IOException e) {
25             System.out.println(" No se pudo leer el archivo" + e.getMessage());
26             return null;
27         }
28         return sb.toString();
29     }
30 }
31

```

leerArchivo :El método leer(String ruta) actúa como el componente de E/S (Entrada/Salida) encargado de cargar el código fuente desde el almacenamiento secundario hacia la memoria principal (RAM), preparando la cadena de caracteres completa que alimentará la etapa de escaneo del analizador léxico.

Clase Generar DOT

```

25 public String generarDot() {
26     StringBuilder sb = new StringBuilder();
27     sb.append("digraph AFD_PromptZal {\n");
28     sb.append("    rankdir=LR;\n");
29     sb.append("    nodesep=0.7;\n");
30     sb.append("    ranksep=1.1;\n");
31     sb.append("    size=\"16,12\";\n");
32     sb.append("    dpi=150;\n");
33     sb.append("    node [shape=circle, fontname=\"Helvetica\", fontsize=11];\n\n");
34
35     sb.append("    inicio [shape=point];\n");
36     sb.append("    inicio -> ").append(tabla.getEstadoInicial()).append(";\n\n");
37
38     for (String estado : tabla.getAceptacion().keySet()) {
39         String token = tabla.getAceptacion().get(estado);
40         sb.append("    ").append(estado)
41             .append(" [shape=doublecircle, label=\"");
42             .append(estado)
43             .append("\n");
44             .append(token).append("\n");
45     }
46
47     sb.append("    ").append(tabla.getEstadoError())
48         .append(" [shape=circle, style=filled, fillcolor=lightcoral];\n\n");
49
50     // NUEVO: agrupar transiciones que van del mismo origen al mismo destino,
51     // combinando sus etiquetas en una sola flecha (ej. "L, D" en vez de dos flechas separadas)
52     java.util.Map<String, java.util.List<String>> agrupadas = new java.util.LinkedHashMap<>();
53     for (TablaTransiciones.Transicion t : tabla.getTransiciones()) {
54         String clave = t.origen + "->" + t.destino;
55         agrupadas.computeIfAbsent(clave, k -> new java.util.ArrayList<>()).add(t.simbolo);
56     }
57
58     for (java.util.Map.Entry<String, java.util.List<String>> entrada : agrupadas.entrySet()) {
59         String[] partes = entrada.getKey().split("->");
60         String etiquetaCombinada = String.join(", ", entrada.getValue());
61         sb.append("    ").append(partes[0])
62             .append(" -> ").append(partes[1])
63             .append(" [label=\"");
64             .append(etiquetaCombinada).append("\n");
65     }
66
67     sb.append("}\n");
68     return sb.toString();
69 }

```

generarDot: El método `generarDot()` es el componente encargado de serializar la estructura lógica de la tabla de transiciones en un script formal con sintaxis **Graphviz (DOT)**, permitiendo la renderización gráfica del Autómata Finito Determinista (AFD).

```

70 public void generarImagen(String rutaSalidaSinExtension) throws IOException, InterruptedException {
71     String dot = generarDot();
72     Path archivoDot = Path.of(rutaSalidaSinExtension + ".dot");
73     Files.writeString(archivoDot, dot);
74
75     ProcessBuilder pb = new ProcessBuilder(
76         "dot", "-Tpng", archivoDot.toString(),
77         "-O", rutaSalidaSinExtension + ".png"
78     );
79     pb.redirectErrorStream(true);
80     Process proceso = pb.start();
81
82     // Leer la salida para evitar que el proceso se bloquee, y para depurar si falla
83     String salida = new String(proceso.getInputStream().readAllBytes());
84     int codigo = proceso.waitFor();
85
86     if (codigo != 0) {
87         throw new IOException("Graphviz fallo al generar la imagen. Salida: " + salida);
88     }
89 }
90 }

```

generarImagen: El método `generarImagen(String rutaSalidaSinExtension)` automatiza la compilación del script DOT hacia un formato gráfico vectorial o de mapa de bits (PNG), invocando externamente la herramienta **Graphviz** mediante la gestión de subprocesos del sistema operativo.

Clase Tabla de Transiciones

```

27
28 private final List<Transicion> transiciones;
29 private final Map<String, String> aceptacion; // estado -> token que emite
30 private final String estadoInicial;
31 private final String estadoError;
32
33 public TablaTransiciones() {
34     this.transiciones = new ArrayList<>();
35     this.aceptacion = new LinkedHashMap<>();
36     this.estadoInicial = "q0";
37     this.estadoError = "qerr";
38     construir();
39 }

```

TablaTransiciones: encapsula la 5-tupla formal del Autómata Finito Determinista (AFD), modelando los estados, el alfabeto de entrada, la función de transición $\delta(q, \sigma)$, el estado inicial y los estados de aceptación con sus respectivos tokens asociados.

Clase Lexer

Esta clase es la más importante de todo el código debido que, aquí es donde se comparan todos los valores o tipos de tokens que existen, dentro de la lógica de esta clase se maneja la clasificación y de ella depende que se haga una buena identificación de cada tipo de token.

```
//Se recorre todo el código y no se detiene ante errores
public void analizar() {
    while (!esFinal()) {
        char c = actual();

        if (c == ' ' || c == '\t' || c == '\r' || c == '\n')
            avanzar();
        } else if (c == '/' && siguiente() == '/') {
            saltarComentarioLinea();
        } else if (c == '/' && siguiente() == '*') {
            saltarComentarioBloque();
        } else if (c == '@') {
            leerDirectiva();
        } else if (esLetra(c) || c == '_') {
            leerPalabra();
        } else if (esDigito(c)) {
            leerNumero();
        } else if (c == '"') {
            leerCadena();
        } else {
            leerSimboloOError();
        }
    }
}
```

analizar : El método usa un ciclo while para recorrer el texto hasta que se llega al final del contenido. Adentro, se obtiene el carácter actual y se evalúa con una estructura de condiciones if y else if. Si el carácter es un espacio en blanco, tabulación o salto de línea, simplemente avanza al siguiente. Si detecta barras, verifica el siguiente carácter para llamar a los métodos que saltan comentarios de línea o de bloque.


```

173 //lectura de simbolos y errores
174 private void leerSimboloError() {
175     int filaInicio = fila;
176     int colInicio = columna;
177     char c = actual();
178
179     switch (c) {
180         case '=' -> {
181             avanzar();
182             agregarToken("=", TipoToken.OP ASIGNACION, filaInicio, colInicio);
183         }
184         case '+' -> {
185             avanzar();
186             agregarToken("+", TipoToken.OP CONCATENACION, filaInicio, colInicio);
187         }
188         case '{', '}', '(', ')', '<' -> {
189             avanzar();
190             agregarToken(String.valueOf(c), TipoToken.DELIMITADOR, filaInicio, colInicio);
191         }
192         case '-' -> {
193             if (siguiente() == '>') {
194                 avanzar();
195                 avanzar();
196                 agregarToken(">", TipoToken.CONECTOR, filaInicio, colInicio);
197             } else {
198                 avanzar();
199                 errores.add(new ErrorLexico("-", "Caracter no reconocido (se esperaba '>')", filaInicio, colIn
200             }
201         }
202         default -> {
203             avanzar();
204             errores.add(new ErrorLexico(String.valueOf(c), "Caracter no reconocido", filaInicio, colInicio));
205         }
206     }
207 }
208

```

leerSimboloError: El método actúa como la subrutina de reconocimiento de operadores, delimitadores y manejo de excepciones léxicas, procesando aquellos caracteres puntuales que no forman identificadores o literales extensos.

Captura de coordenadas léxicas: Registra de forma inmediata los punteros de posición `filaInicio` e `colInicio` antes de consumir el carácter mediante `actual()`, asegurando una trazabilidad espacial exacta para la emisión de tokens o reportes de error.

Captura y recuperación ante errores léxicos: Tanto la rama `else` del conector como la cláusula `default` canalizan cualquier carácter no reconocido fuera del alfabeto formal hacia la colección errores instanciando un `ErrorLexico`. Esto evita la interrupción abrupta del flujo (panic mode parcial), consumiendo el símbolo inválido con `avanzar()` para permitir que el escáner continúe con el resto de la entrada.

Clase Generar Reporte

```
public void generarReporteEstadisticas(List<Token> tokens, List<ErrorLexico> errores,
                                     int totalLineas, String ruta) throws IOException {
    // Conteo de frecuencia por tipo de token, en el orden en que aparece cada tipo por primera vez
    Map<String, Integer> frecuencia = new LinkedHashMap<>();
    for (Token t : tokens) {
        String tipo = t.getTipo().toString();
        frecuencia.put(tipo, frecuencia.getOrDefault(tipo, 0) + 1);
    }

    StringBuilder html = new StringBuilder();
    html.append("<!DOCTYPE html>\n<html lang='es'>\n<head>\n");
    html.append("<meta charset='UTF-8'>\n<title>Reporte de Estadísticas - PromptZal</title>\n");
    html.append(estilosHTML());
    html.append("</head>\n<body>\n");

    html.append("<div class='container'>\n");
    html.append("<h1>Reporte de Estadísticas</h1>\n");
    html.append("<p>Resumen general del análisis léxico</p>\n");

    // Resumen de totales
    html.append("<div class='table-wrapper'>\n");
    html.append("<table>\n<tr><th style='background-color: #DDA0DD; color: #1e293b;'>Métrica</th><th style='backg");
    html.append("<tr><td>Total de tokens reconocidos</td><td><span>")
        .append(tokens.size())
        .append("</span></td></tr>\n");
    html.append("<tr><td>Total de líneas analizadas</td><td><span>")
        .append(totalLineas)
        .append("</span></td></tr>\n");
    html.append("<tr><td>Total de errores léxicos</td><td><span class='error-count'>")
        .append(errores.size())
        .append("</span></td></tr>\n");
    html.append("</table>\n</div>\n");
}
```

generarReporteEstadisticas: este método es la nueva tabla que se agregó a la clase y genera una tabla HTML que representa las estadísticas del análisis, desde la cantidad de tokens generada, cantidad de errores, lo clasifica por tipos, muestra su posición y agrupación de métricas léxicas.

```
private void escribirArchivo(String ruta, String contenido) throws IOException {
    try (FileWriter writer = new FileWriter(ruta)) {
        writer.write(contenido);
    }
}
```

escribirArchivo: encargada de volcar cadenas de texto plano (como los reportes HTML generados) directamente hacia el almacenamiento secundario.

MANEJO DEL ERROR

El analizador implementa un mecanismo de recuperación de errores a nivel de carácter: cuando el AFD no encuentra una transición válida desde el estado actual para el símbolo leído, en vez de detener el análisis, reporta el error y continúa desde el estado inicial (q0), descartando la menor cantidad de entrada posible. Esto permite que un archivo con múltiples errores léxicos los reporte todos en una sola corrida, en vez de detenerse en el primero encontrado.

Cómo se representa en el AFD

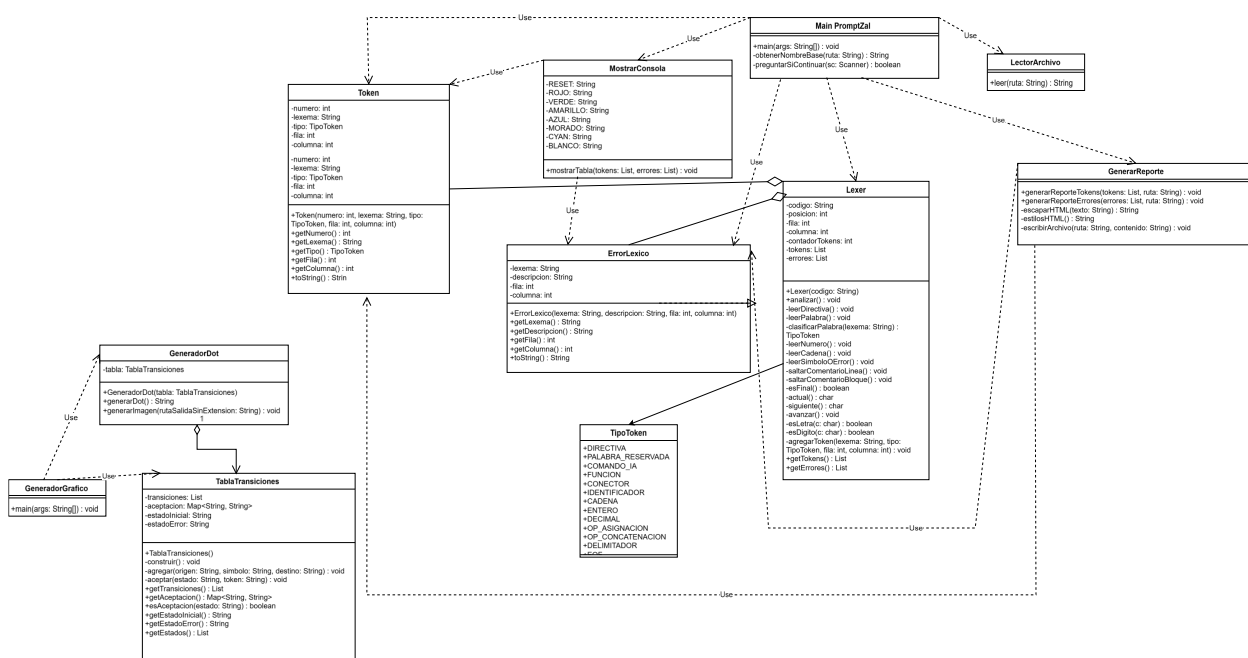
Cualquier estado que reciba un símbolo para el cual no existe transición definida en la tabla, se dirige al estado trampa qerr. En la tabla de transiciones, esto corresponde a las celdas marcadas explícitamente hacia qerr .

Categorías de error que el analizador detecta

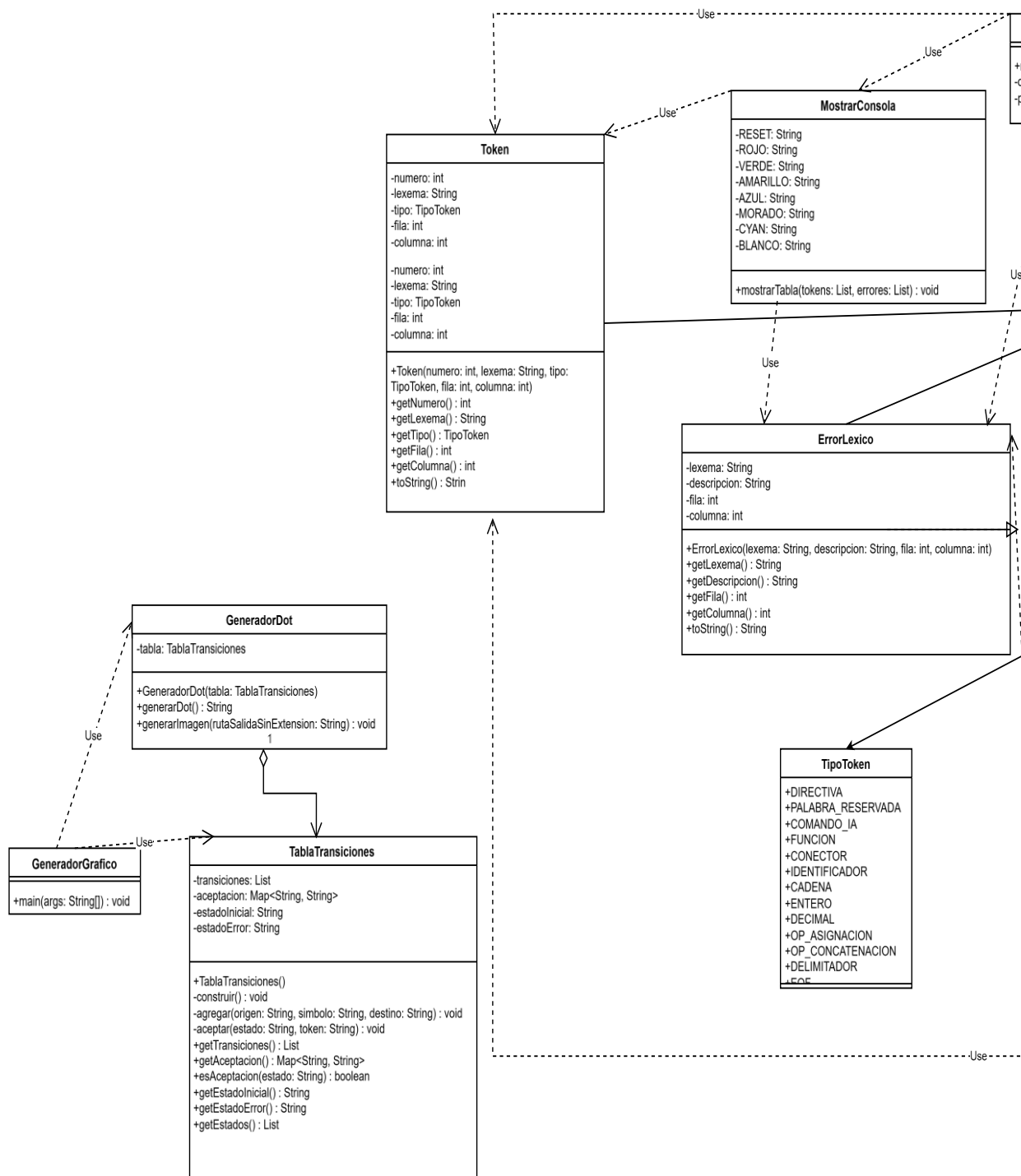
Categoría	Ejemplo	Dónde ocurre en el AFD
Carácter no reconocido	\$, %, ~, €, , , etc	q0 → qerr
Directiva no reconocida	@invalido	Se acepta como palabra pero se valida contra la lista de directivas válidas fuera del AFD
Decimal mal formado	10. (sin dígito después del punto)	q5 → qerr
Cadena sin cerrar	"texto sin comilla final	q7 → qerr (al llegar \n o EOF)
Comentario de bloque sin cerrar	/* texto sin cierre	q15/q16 sin llegar nunca a q0 (EOF alcanzado)
Símbolo incompleto	- sin > siguiente	q11 → qerr

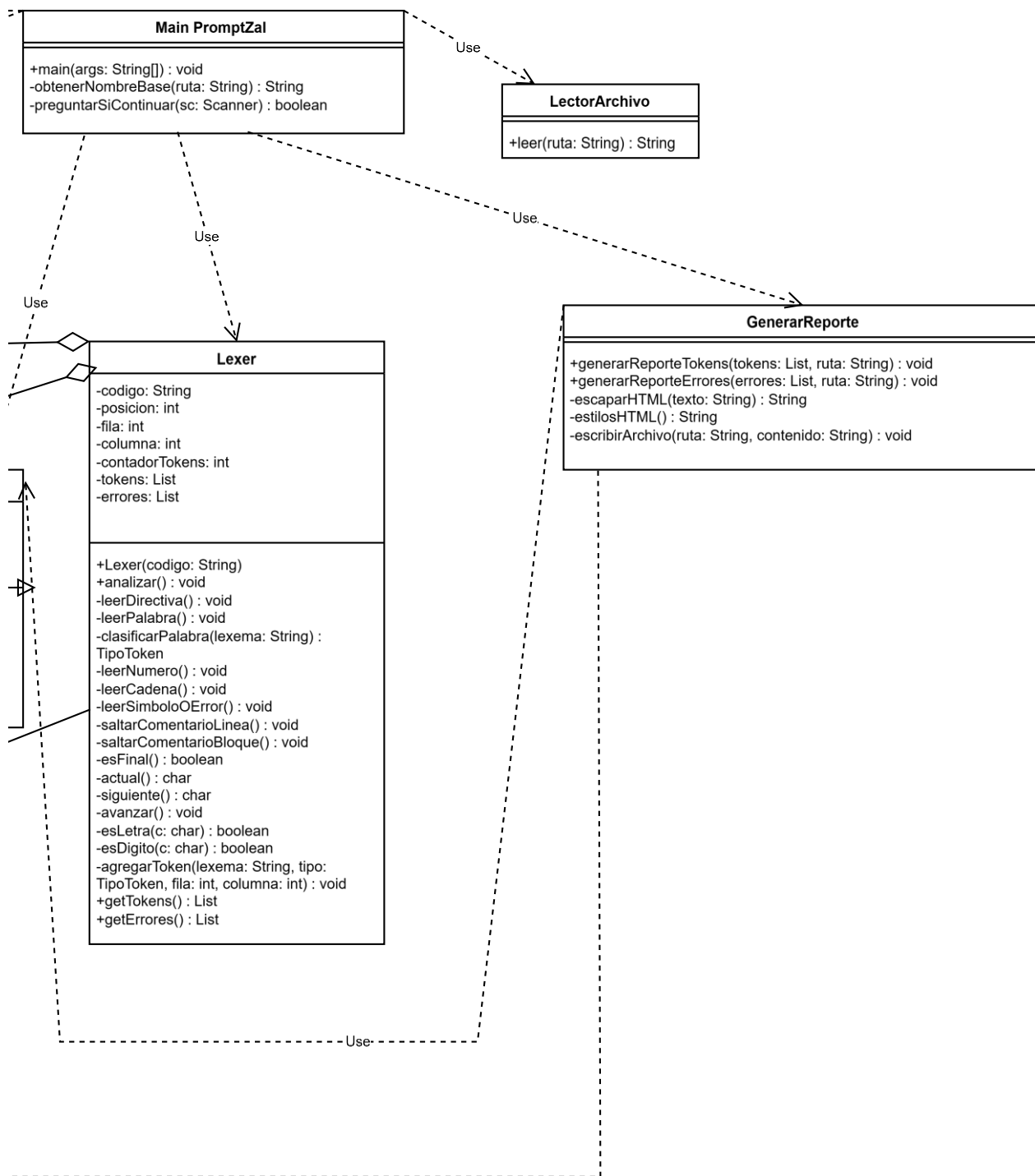
DIAGRAMA UML

Diagrama UML que refleja la estructura del código y como están conectadas todas las clases y como se relacionan entre sí, el diagrama corresponde únicamente al backend del proyecto.



Se separa en dos para poder visualizar mejor





CONCLUSION

El desarrollo de este proyecto permitió transformar el analizador léxico exploratorio de la Práctica 1 en un analizador formalmente fundamentado, respaldado por un autómata finito determinista y su gramática regular equivalente. El proceso de diseño evidenció que, si bien el AFD y la gramática son representaciones matemáticamente equivalentes del mismo lenguaje regular, su implementación en código responde a propósitos distintos: el Lexer prioriza la eficiencia y claridad del reconocimiento manual carácter por carácter, mientras que la tabla de transiciones prioriza la capacidad de representarse y visualizarse como estructura de datos.

La incorporación de recuperación de errores robusta, validada empíricamente con casos de prueba que combinan tokens válidos y errores léxicos diversos, demuestra que el analizador cumple con el objetivo de reportar de forma exhaustiva las inconsistencias léxicas de un archivo en una sola ejecución. Asimismo, la interfaz gráfica desarrollada en Java Swing y los reportes generados en HTML permiten comunicar los resultados del análisis de forma clara y accesible, cumpliendo con el objetivo de que el analizador esté, además de bien fundamentado, bien presentado.